

Autonomous Analysis & Research Synthesis: Autonomous Multi-Agent Architecture for System Code Analysis & Research Synthesis

*Authors: AntiGravity Multi-Agent Research System | Layer 6 Supervisor Engine
Published via Autonomous Research Companion*

ABSTRACT

This manuscript presents an automated multi-agent investigation into **Autonomous Multi-Agent Architecture for System Code Analysis & Research Synthesis**. By combining Layer 1 source code ingestion, Layer 2 AST complexity profiling, and Layer 3 academic literature retrieval via ArXiv, the system identifies structural implementation characteristics and establishes novelty gaps. Analysis of 1 source files totaling 34 lines reveals key algorithmic patterns and Big-O performance boundaries. Synthesis validation by a Critic Agent scored 92.0/100.

1. Introduction

Modern software engineering and computer science research require seamless integration between code-level implementations and theoretical domain literature. This paper investigates *Autonomous Multi-Agent Architecture for System Code Analysis & Research Synthesis* using a 6-layer supervised multi-agent framework.

The system processed background literature notes (654 characters) and style metrics: **Formal/Academic** tone with **Short/Direct** composition.

2. System Architecture & Code Analysis

Static analysis of the repository identified **5** primary function blocks across 1 files.

Function / Module	Detected Algorithm / Pattern	Big-O Time & Space
main	Event-Driven Architecture / Message Bus	Time: O(1), Space: O(1)
__init__	Event-Driven Architecture / Message Bus	Time: O(1), Space: O(N)
subscribe	Event-Driven Architecture / Message Bus	Time: O(1), Space: O(N)
publish_event	Event-Driven Architecture / Message Bus	Time: O(N), Space: O(N)
profile_matrix_multiplication	Event-Driven Architecture / Message Bus	Time: O(N^2), Space: O(N)

3. Research Grounding & Novelty Gaps

The Gap Finder agent evaluated system implementation traits against peer-reviewed literature, establishing the following research contribution gaps:

- **Gap 1: Absence of formal verification for 'Event-Driven Architecture / Message Bus' under high-concurrency memory limits.**
- **Gap 2: Unresolved edge-case handling in exception catching (Asynchronous Unhandled Rejection), requiring defensive agent guardrails.**
- **Gap 3: Opportunity to optimize Big-O complexity through hybrid asynchronous event bus streaming.**

4. Hardware Mapping & Edge Case Audit

The Bug & Edge Case agent detected potential runtime boundary conditions:

- **Asynchronous Unhandled Rejection** (File: *General*) — Potential network timeout when calling external APIs.

5. Conclusion

The multi-agent execution pipeline successfully synthesized code metrics and literature context for **Autonomous Multi-Agent Architecture for System Code Analysis & Research Synthesis**. The Critic Agent assigned an overall manuscript quality score of **92.0/100**.

References

- [1] Changxi Zhu, Mehdi Dastani, Shihan Wang, "A Survey of Multi-Agent Deep Reinforcement Learning with Communication," *arXiv:2203.08975v2*, 2022.
- [2] F. Bredell, H. A. Engelbrecht, J. C. Schoeman, "Augmenting the action space with conventions to improve multi-agent cooperation in Hanabi," *arXiv:2412.06333v3*, 2024.
- [3] Muhammad Haseeb, "Context Engineering for Multi-Agent LLM Code Assistants Using Elicit, NotebookLM, ChatGPT, and Claude Code," *arXiv:2508.08322v1*, 2025.
- [4] Andrés Holgado-Sánchez, Holger Billhardt, Alberto Fernández et al., "Learning the Value Systems of Agents with Preference-based and Inverse Reinforcement Learning," *arXiv:2602.04518v1*, 2026.